Live Demos — Facilitator Guide

For: Tim Smith · Companion to the five progressive live demos (`live-demos/`)

This guide is for **setting up and running the live-demo portion** of the workshop. The demos are the "do/observe" counterpart to the deck: five Dockerized levels that each add one capability and light up more of the 7-layer stack. Attendees paste a shared OpenAI key and watch (or drive) each level.

---

1. What you're running

One Streamlit app, five pages (Levels 1 → 5), in a single Docker container:

Level	Demonstrates	Layers	New vs. previous level
1 · Chatbot	system prompt + message	1, 3	the baseline
2 · Memory + guardrails	conversation memory; scoped support bot; guardrail you can toggle	1, 3, 7	memory + guardrails
3 · Context engineering	retrieval + assembled context; grounded, cited answers	4, 6	an information store
4 · MCP + tools	tool-using agent over an MCP-style server	2, 5	tools / the ability to act
5 · A2A + governance	multi-agent collaboration under RBAC, approval gate, audit log	2, 7	multiple agents + governance

Architecture, kept deliberately simple: a single container; retrieval uses an **in-memory** index over OpenAI embeddings (no external database); the MCP server and the multiple agents run **in-process** (the standalone real-protocol MCP server is in `mcp-lab/`). The only external dependency is the OpenAI API.

---

2. Prerequisites

- **Docker Desktop** (Mac/Windows/Linux) — https://www.docker.com/products/docker-desktop/
- **One OpenAI API key** that you provision and hand out (see §5).
- That's it for attendees who run it themselves. If they'd rather just watch, you run it and screen-share.

---

3. Run it (one command)

```
cd live-demos
docker compose up --build       # first build ~1-2 min; later runs are instant
```

Open **http://localhost:8501**, paste the OpenAI key in the sidebar, and walk Levels 1 → 5.

- Stop: `Ctrl-C`. Remove the container: `docker compose down`.
- No Docker? `pip install -r requirements.txt && streamlit run app.py`.
- Port 8501 busy? Edit the port mapping in `docker-compose.yml` (e.g., `"8600:8501"`).

---

4. Running each level live (what to point out)

Level 1 — Chatbot. Send a message; expand "Exactly what is sent to the model" to show it's just a system prompt + a user turn. Change the system prompt ("answer only in haiku") and resend. Then send a follow-up and note it has *no memory*. Takeaway: a bare chatbot is configuration over a model call.

Level 2 — Memory + guardrails. Ask a Northwind support question, then a follow-up like "and how do I undo that?" — memory lets it follow along (open the memory expander). Then ask something off-topic ("write me a poem") with **guardrails ON** (blocked) vs **OFF** (it wanders). Takeaway: memory and guardrails are additions you choose, and governance starts here.

Level 3 — Context engineering. Ask "how long do enterprise customers have for a refund?" Show the retrieved chunks, then the **assembled prompt** (the engineered context), then the grounded, cited answer. Tick "show the ungrounded answer" to contrast. Takeaway: retrieval quality dominates output quality.

Level 4 — MCP + tools. Show the server's tool catalog, then run "Is order 4471 within the refund window?" Watch the client↔server tool-call trace (request/response) before the final answer. Takeaway: adding a capability is adding a *tool to a server*, not changing the model.

Level 5 — A2A + governance. Run the refund workflow for order 4471. Show the A2A message timeline, the RBAC table (only the action agent may issue a refund — expand "prove RBAC" to see a read-side write blocked), the **approval gate** (nothing executes until you click Approve), and the audit log. Takeaway: autonomy made safe with Layer-7 controls.

Each page shows a **"layers in play"** badge so the tie back to the deck's stack is explicit.

---

5. The OpenAI key — setup & safety

- Create a **dedicated, project-scoped key** with a **hard budget cap** ($25–50 is plenty).
- The demos default to `gpt-4o-mini`, cap output tokens, and enforce a **per-session request limit** — so a shared key can't run up a surprise bill.
- The key is entered in the sidebar and held **in browser session memory only** — never written to disk or logged.
- **Revoke the key right after the workshop.**
- Optional: to skip the paste, set `openai_api_key` in `.streamlit/secrets.toml` (see `live-demos/.streamlit` patterns), but pasting is the default.

Rough cost: each level is a handful of small `gpt-4o-mini` calls; a room of faculty clicking through all five levels is typically a few dollars, not tens.

---

6. Distributing to attendees

Three options, easiest first:

1. **You drive, they watch** (screen-share) — zero attendee setup; best for the live session.
2. **Attendees run locally** — they need Docker Desktop + the key; hand out the repo URL and §3.
3. **One shared instance** — run the container on a laptop/VM and share the URL on the room network (it's a plain Streamlit app on a port). Add a passphrase if you expose it widely.

---

7. Troubleshooting

Symptom	Fix
"OpenAI call failed: …auth…"	Wrong/empty key — re-paste in the sidebar.
First model reply is slow	Normal cold start; subsequent calls are quick.
"Per-session request limit reached"	A safety cap — refresh the page to reset.
Port 8501 in use	Change the mapping in `docker-compose.yml`.
Build is slow the first time	One-time image build; cached afterward.

---

8. How this fits the rest of the kit

- **Deck** — the "Five live demos" slide near the end points here.
- **Companion** — Part 10 describes each level and what to observe.
- **`workshop-labs/`** — self-driven hands-on *exercises* (Streamlit + a browser prompt lab); these live demos are the guided *observe-the-progression* counterpart.
- **`mcp-lab/`** — the real MCP protocol over Docker, for anyone who wants Level 4 at protocol depth.